

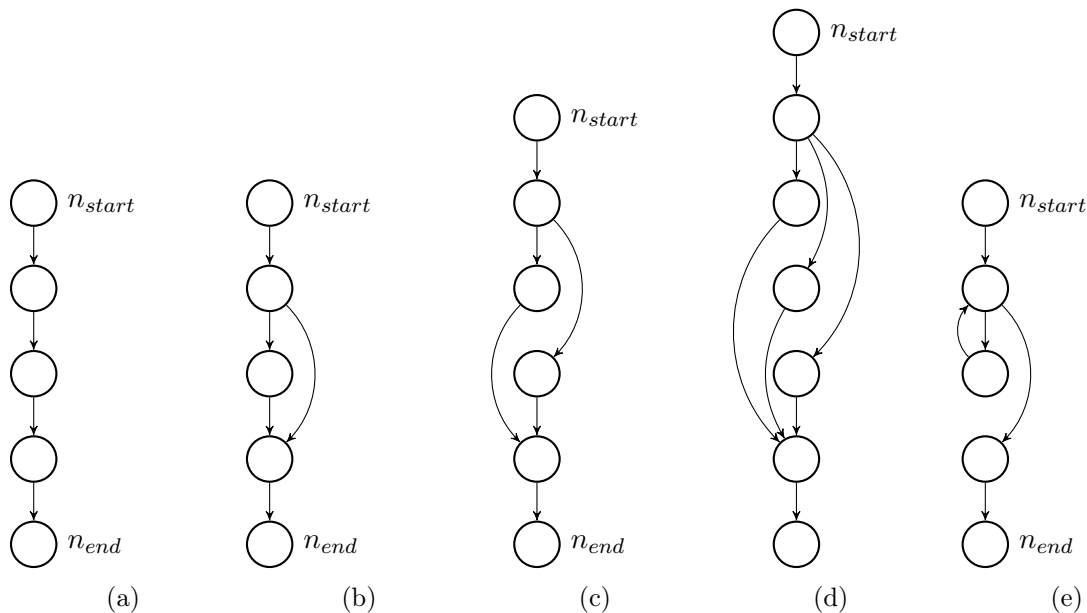
Softwarekonstruktion WS 15/16 – Übung 4

Ausgabe: 07.12.2015, Abgabe: 04.01.2016 12:00 Uhr

Präsenzübung

4.1 Zyklomatische Komplexität

1. Bestimmen Sie die zyklomatische Komplexität folgender Kontrollflussgraphen.
2. Um welches Konstrukt könnte es sich jeweils handeln?
3. Wie wirken sich Sequenzen, bedingte Anweisungen, Fallunterscheidungen und Schleifen auf die zyklomatische Komplexität aus?



Erinnerung: $ZK = |E| - |V| + |P| + 1$

- (a): Sequenz: 1
- (b): Bedingte Anweisung (**if-then**): 2
- (c): Bedingte Anweisung (**if-then-else**): 2
- (d): Fallunterscheidung (**switch-case**): 3
- (e): Schleife (**while-do**): 2

Auswirkungen:

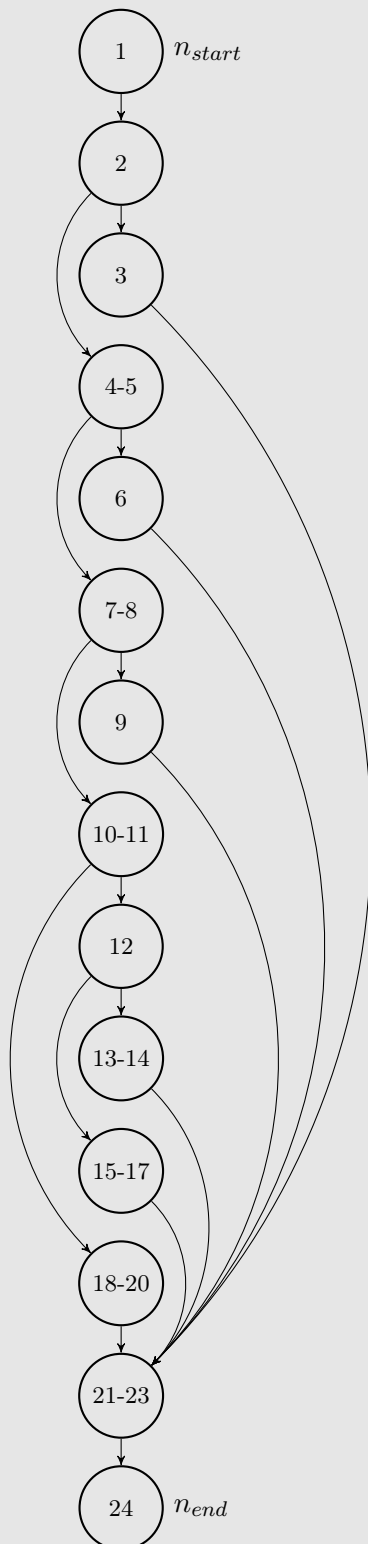
- Sequenzen beeinflussen die zykl. Komplexität nicht.

- Bedingte Anweisungen (**if-then-else**) erhöhen die zykl. Komplexität um 1.
- Fallunterscheidungen (**switch-case**) erhöhen die zykl. Komplexität in Abhängigkeit der unterschiedenen Fälle.
- Schleifen erhöhen die zykl. Komplexität um 1.

4.2 Zyklomatische Komplexität eines Quelltextes.

Zeichnen Sie den Kontrollflussgraphen folgender Funktion und bestimmen Sie die zyklomatische Komplexität.

```
1 void output(int selection , int y){
2     if (selection==1) {
3         printf("\n_1");
4     } else {
5         if (selection==2) {
6             printf("\n_2");
7         } else {
8             if (selection==3) {
9                 printf("\n_3");
10            } else {
11                if (selection==4) {
12                    if (y==0) {
13                        printf("\n_4");
14                        printf("\n_5");
15                    } else {
16                        printf("\n_6");
17                    }
18                } else {
19                    printf("\n_7");
20                }
21            }
22        }
23    }
24 }
```



Die Funktion hat eine zyklomatische Komplexität von $|E| - |V| + |P| + 1 = 18 - 14 + 1 + 1 = 6$.
Dabei sei E die Menge der Kanten, V die Menge der Knoten und P die Menge der End-

knoten.

Alternativ: $\# \text{Verzweigungen} + 1 = 5 + 1 = 6$.

4.3 CK-Metriken

Geben Sie Definitionen und Beispiele für die folgenden Metriken an.

1. WMC
2. DIT
3. NOC

Lösungsvorschlag

1. Weighted Methods per Class (WMC): Anzahl der Methoden der Klasse, gewichtet durch Komplexität: $WMC = \sum_1^n C(i)$, wobei n die Anzahl der Methoden der Klasse und $C(i)$ die Komplexität von Methode i bezeichnet.
2. Depth of Inheritance Tree (DIT): Maximale Tiefe Klasse in der Generalisierungshierarchie
3. Number of Children (NOC): Anzahl direkter Unterklassen

4.4 UCP-Schätzung

Ein Softwareunternehmen hat eine Anfrage zur Entwicklung einer Software vorliegen. Die zu erstellende Software soll mit verschiedenen Fremdsystemen interagieren. Zwei Fremdsysteme sollen mittels einer wohldefinierten API mit der zu erstellenden Software kommunizieren können, außerdem werden zwei Fremdsysteme über REST-Schnittstellen angesprochen. Eine GUI ist nicht vorgesehen. Im Rahmen einer ersten Anforderungsanalyse wurden 5 Use Cases ermittelt. Neben zwei einfachen Use Cases wurde einer mit sechs Transaktionen, einer mit acht und einer mit zehn Transaktionen identifiziert.

Technische Komplexität und Umgebung wurden bereits wie folgt charakterisiert:

F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}
0	2	0	1	0	5	0	3	5	0	0	1	2

E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8
1	4	3	1	2	0	5	1

Zur Plausibilisierung eines Angebots soll nun der Projektumfang nach der Use-Case-Point-Methode (wie in der Vorlesung vorgestellt) abgeschätzt werden. Aus vergangenen Projekten wird ein Aufwand von 22 Personenstunden pro Use-Case-Point angenommen. Geben Sie die Werte für $UUCW$, UAW , TCF und ECF an und den errechneten Gesamtaufwand in Personenstunden an.

Wir beziehen uns auf die Abschlussarbeit von Karner: <http://www.bfpug.com.br/Artigos/UCP/Karner%20-%20Resource%20Estimation%20for%20objectory%20Projects.doc>

- Unbereinigte Use Case Gewichte:

$$UUCW = 2 * 5 + 1 * 10 + 2 * 15 = 50$$

- Unbereinigte Aktor Gewichte:

$$UAW = 2 * 1 + 2 * 2 + 0 * 3 = 6$$

- Technischer Komplexitätsfaktor:

F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	TF
0	2	0	1	0	5	0	3	5	0	0	1	2	
2.0	1.0	1.0	1.0	1.0	0.5	0.5	2.0	1.0	1.0	1.0	1.0	1.0	
0	2.0	0.0	1.0	0.0	2.5	0.0	6.0	5.0	0.0	0.0	1.0	2.0	19.5

$$TCF = 0.6 + (TF/100) = 0.795$$

- Umgebungskomplexitätsfaktor:

E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	EF
1	4	3	1	2	0	5	1	
1.5	-1.0	0.5	0.5	1.0	1.0	-1.0	2.0	
1.5	-4.0	1.5	0.5	2.0	0.0	-5.0	2.0	-1.5

$$ECF = 1.4 + (-0.03 * EF) = 1.445$$

- Use-Case-Points:

$$UCP = (UUCW + UAW) * TCF * ECF = 64.3314$$

- Aufwand in Personenstunden:

$$F = 22h$$

$$UCP \cdot F = 1415.291h$$

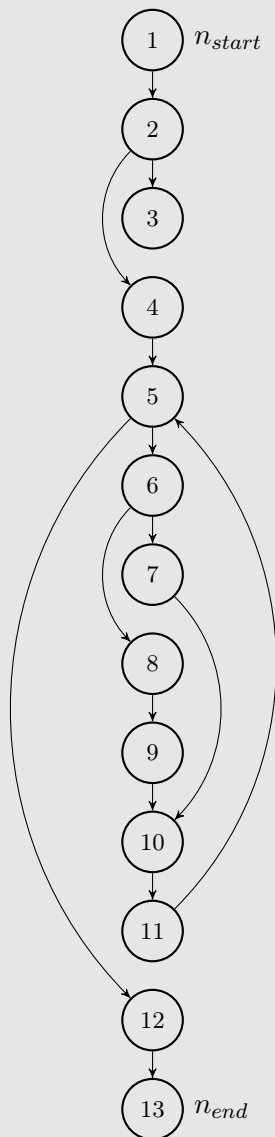
Hausübung

Hinweis zu dieser Hausübung: Die Aufgabe 4.7 muss von jedem Teilnehmer in Moodle durchgeführt und abgeschlossen werden, kann jedoch in Gruppen besprochen werden. Die übrigen Aufgaben müssen auf Papier in den Briefkasten 21, OH 12 eingeworfen werden (Gruppenabgabe: bis zu vier Personen einer Übungsgruppe).

4.5 Zyklomatische Komplexität (14 Punkte)

Betrachten Sie die folgende Funktion. Zeichnen Sie den Kontrollflussgraphen der Funktion und bestimmen Sie die zyklomatische Komplexität.

```
1 int ggT(int a, int b) {  
2     if (a == 0) {  
3         return b;  
4     }  
5     while(b != 0) {  
6         if (a > b) {  
7             a = a - b;  
8         } else {  
9             b = b - a;  
10        }  
11    }  
12    return a;  
13 }
```



Die Funktion hat eine zyklomatische Komplexität von $|E| - |V| + |P| + 1 = 14 - 13 + 2 + 1 = 4$. Dabei sei E die Menge der Kanten, V die Menge der Knoten und P die Menge der Endknoten.

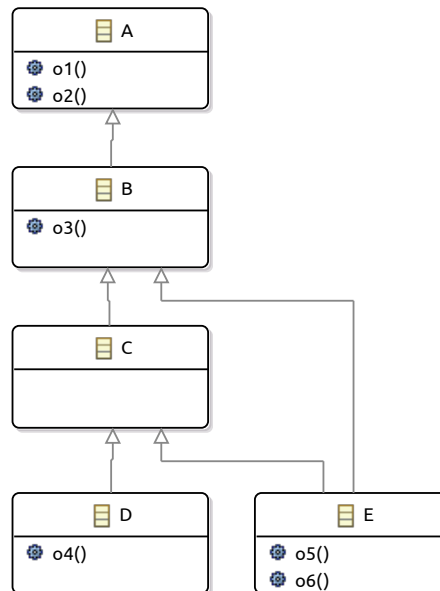
Alternativ: $\# \text{Verzweigungen} + 1 = 3 + 1 = 4$.

Bewertungsschema

- 8 Punkte für den Kontrollflussgraphen.
- 6 Punkte für die zyklomatische Komplexität.

4.6 CK-Metriken (6 Punkte)

Bestimmen Sie für jede im folgenden Klassendiagramm dargestellte Klasse die drei Metriken *WMC*, *DIT* und *NOC*. Nehmen Sie für jede Methode i an, dass $C(i) = 1$ gilt.



Klasse	WMC	DIT	NOC
A	2	0	1
B	1	1	2
C	0	2	2
D	1	3	0
E	2	3	0

Bewertungsschema

2 Punkte je Metrik (Spalte).

4.7 Bestimmung von Äquivalenzklassen (6 Punkte)

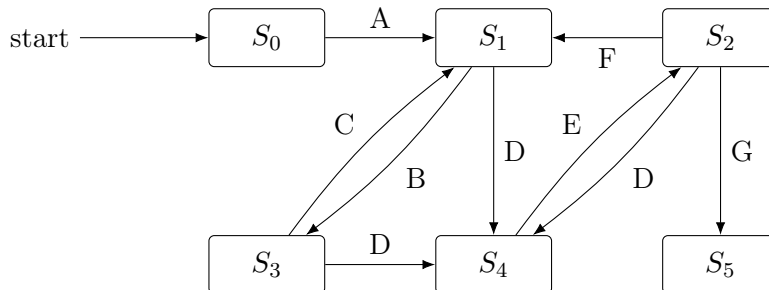
Diese Aufgabe ist in Moodle zu bearbeiten. Die Bearbeitung *dieser* Aufgabe ist möglich bis zum 5. Januar 2016 4:00 Uhr.

Bewertungsschema

Die Bewertung findet in Moodle statt.

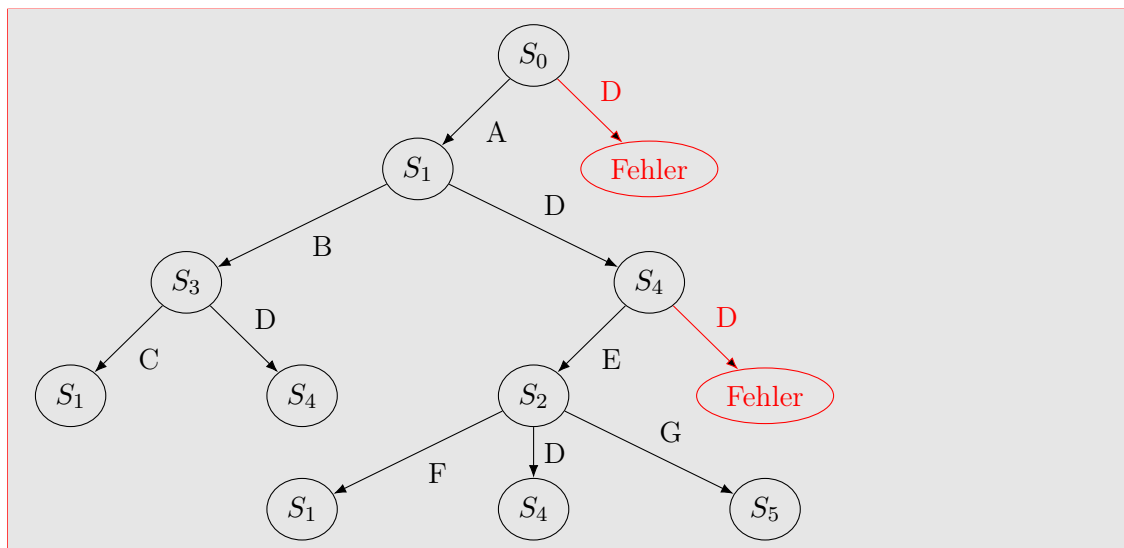
4.8 Zustandsbasiertes Testen (14 Punkte)

Gegeben ist folgendes Zustandsdiagramm.



Die Aktion D führt in den Zuständen S_0 und S_4 zu einem Fehler.

- Erstellen Sie den Übergangsbaum inklusive der Fehlerzustände.



- Geben Sie eine möglichst kleine Menge von Funktionssequenzen an, die im Rahmen eines Zustandskonformanztests eine Zustandsüberdeckung von 100% erzielt.

Werden Funktionssequenzen aus dem Übergangsbaum abgeleitet, so dass jeder Zustand mindestens einmal abgedeckt wird, ergibt sich beispielsweise:

- $(S_0) - A - (S_1) - B - (S_3) - C - (S_1)$
- $(S_0) - A - (S_1) - D - (S_3) - E - (S_1) - G - (S_5)$

Die kleinstmögliche Menge besteht jedoch aus lediglich einer Funktionssequenz, beispielsweise deckt folgende Sequenz alle Zustände ab:

- $(S_0) - A - (S_1) - B - (S_3) - D - (S_4) - E - (S_2) - G - (S_5)$

- Geben Sie eine möglichst kleine Menge von Funktionssequenzen an, die im Rahmen eines Zustandskonformanztests eine Zustandsübergangsüberdeckung von 100% erzielt?

Werden wieder Funktionssequenzen aus dem Übergangsbaum abgeleitet, wird nun für jedes Blatt des Baumes ein Pfad von der Wurzel des Baumes zum Blatt verwendet, also:

- $(S_0) - A - (S_1) - B - (S_3) - C - (S_1)$
- $(S_0) - A - (S_1) - B - (S_3) - D - (S_4)$
- $(S_0) - A - (S_1) - D - (S_4) - E - (S_2) - F - (S_1)$
- $(S_0) - A - (S_1) - D - (S_4) - E - (S_2) - D - (S_4)$
- $(S_0) - A - (S_1) - D - (S_4) - E - (S_2) - G - (S_5)$

Jedoch wurde wieder nach einer möglichst kleinen Menge gefragt. Die folgende Funktionssequenz bildet eine möglichst kleine (nämlich eine einelementige) Menge, die alle Zustandsübergänge mindestens einmal anregt:

- $(S_0) - A - (S_1) - B - (S_3) - C - (S_1) - D - (S_4) - E - (S_2) - F - (S_1) - D - (S_4) - E - (S_2) - D - (S_4) - E - (S_2) - G - (S_5)$

4. Welche Funktionssequenz/en wird/werden zusätzlich für eine Zustandsüberdeckung von 100% im Rahmen eines Zustandsrobustheitstests benötigt?

Zusätzlich zu der Funktionssequenz aus Teilaufgabe 2 wird beispielsweise benötigt:

- $(S_0) - D - (\text{Fehler})$

Es wird dabei angenommen, dass die Ausführung der Aktion D in S_0 und S_4 in denselben Fehlerzustand führt.

5. Welche Funktionssequenz/en wird/werden zusätzlich für eine Zustandsübergangsüberdeckung von 100% im Rahmen eines Zustandsrobustheitstests benötigt?

Zusätzlich zu der Funktionssequenz aus Teilaufgabe 3 wird beispielsweise benötigt:

- $(S_0) - D - (\text{Fehler})$
- $(S_0) - A - (S_1) - D - (S_4) - D - (\text{Fehler})$

Bewertungsschema

1. 2 Punkte
2. 3 Punkte. 2 Punkte, falls alle Zustände durch mehr als eine Sequenz abgedeckt werden
3. 3 Punkte. 2 Punkte, falls alle Zustandsübergänge durch mehr als eine Sequenz abgedeckt werden
4. 3 Punkte
5. 3 Punkte